

Table of Contents

1. EC2 Instance Setup	2
2. Project Structure Setup	2
3. Application Development	3
4. Dockerization of the Application.....	6
4.1 Web Dockerfile	6
4.2 MySQL Initialization Script.....	6
5. Docker Compose Setup.....	8
6. GitHub Repository Setup	11
7. Jenkins Installation & Configuration	13
8. GitHub Webhook Configuration.....	13
9. Jenkins Pipeline (CI/CD) Implementation	14
9.1 Checkout Code	14
9.2 Linting	14
9.3 Build Docker Images	14
9.4 Unit Tests	14
9.5 Integration + Selenium Tests	14
9.6 Artifact Collection.....	14
9.7 Cleanup	14
10. Pipeline Execution and Success	15
Appendix:.....	17
Dockerfile for WebApp:	17
Db/init.sql:	17
Selenium Dockerfile:	17
Selenium/run_tests.sh	18
test_unit.py.....	19
test_selenium.py.....	19
docker-compose.yml	20
Jenkinsfile:	21

1. EC2 Instance Setup

A fresh Ubuntu EC2 instance was launched on AWS. The following essential tools were installed:

- Docker
- Docker Compose
- Git
- Python3 + pip
- Jenkins

Docker permissions were configured:

```
sudo usermod -aG docker ubuntu
```

```
sudo usermod -aG docker jenkins
```

```
newgrp docker
```

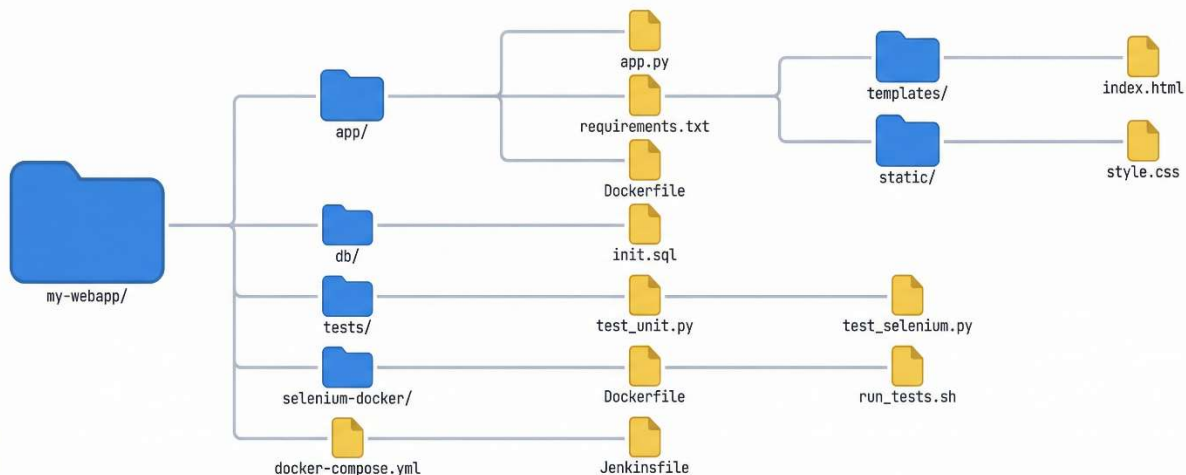
```
sudo systemctl restart jenkins
```

This ensured both the EC2 user and Jenkins had access to Docker.

2. Project Structure Setup

The first step was to define a clean and modular project structure. The following directory layout was created:

Project Structure — my-webapp



This structure separates the web application, database configuration, Selenium testing environment, and CI/CD pipeline definition.

```
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.
```

```
ubuntu@ip-172-31-17-82:~$
```

i-037c19b2f436f218e (theoryassign3ZainabQ)

PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

3. Application Development

3.1 Flask Application

A simple To-Do List web application was implemented using Flask and PyMySQL.

The application supports:

- Viewing tasks
- Adding new tasks
- Deleting tasks
- Interacting with a MySQL database using API endpoints

The UI was created using index.html and style.css.

```
GNU nano 7.2 app.py *  
from flask import Flask, request, jsonify, render_template  
import pymysql  
import os  
  
app = Flask(__name__)  
  
DB_HOST = os.environ.get("DB_HOST", "db")  
DB_USER = os.environ.get("DB_USER", "root")  
DB_PASS = os.environ.get("DB_PASS", "password")  
DB_NAME = os.environ.get("DB_NAME", "tododb")  
  
def get_conn():  
    return pymysql.connect(  
        host=DB_HOST,  
        user=DB_USER,  
        password=DB_PASS,  
        database=DB_NAME,  
        charset='utf8mb4',  
        collation='utf8mb4_unicode_ci')  
  
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/
```

i-037c19b2f436f218e (theoryassign3ZainabQ)

PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

```
GNU nano 7.2 requirements.txt *  
Flask==2.2.5  
pymysql==1.0.3  
pytest==7.3.1  
selenium==4.10.0  
flake8==6.1.0  
requests==2.31.0  
^
```

```
GNU nano 7.2 Dockerfile *
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Loc
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go
```

i-037c19b2f436f218e (theoryassign3ZainabQ)

PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

```
GNU nano 7.2 index.html *
<!DOCTYPE html>
<html>
<head>
  <title>To-Do List</title>
  <link rel="stylesheet" href="{ url_for('static', filename='style.css') }">
</head>
<body>
  <h1>My To-Do List</h1>

  <form id="todo-form">
    <input type="text" id="task-input" placeholder="Enter a task" required>
    <button type="submit">Add</button>
  </form>

  <ul id="todo-list">

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Locati
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To
```

i-037c19b2f436f218e (theoryassign3ZainabQ)

PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

```

ubuntu@ip-172-31-17-82:~$ mkdir webapp
ubuntu@ip-172-31-17-82:~$ cd webapp
ubuntu@ip-172-31-17-82:~/webapp$ mkdir app
ubuntu@ip-172-31-17-82:~/webapp$ mkdir db
ubuntu@ip-172-31-17-82:~/webapp$ mkdir tests
ubuntu@ip-172-31-17-82:~/webapp$ mkdir selenium-docker
ubuntu@ip-172-31-17-82:~/webapp$ ls
app db selenium-docker tests
ubuntu@ip-172-31-17-82:~/webapp$ cd app
ubuntu@ip-172-31-17-82:~/webapp/app$ nano app.py
ubuntu@ip-172-31-17-82:~/webapp/app$ nano models.py
ubuntu@ip-172-31-17-82:~/webapp/app$ nano requirements.txt
ubuntu@ip-172-31-17-82:~/webapp/app$ nano Dockerfile
ubuntu@ip-172-31-17-82:~/webapp/app$ mkdir templates
ubuntu@ip-172-31-17-82:~/webapp/app$ mkdir static
ubuntu@ip-172-31-17-82:~/webapp/app$ cd templates
ubuntu@ip-172-31-17-82:~/webapp/app/templates$ nano index.html
ubuntu@ip-172-31-17-82:~/webapp/app/templates$ cd ..
ubuntu@ip-172-31-17-82:~/webapp/app$ 

```

i-037c19b2f436f218e (theoryassign3ZainabQ)

PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

```

GNU nano 7.2 style.css *
/* General page styling */
body {
    font-family: Arial, sans-serif;
    background: #f4f4f4;
    padding: 40px;
    margin: 0;
    color: #333;
}

/* Main container */
h1 {
    text-align: center;
    color: #444;
    margin-bottom: 30px;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Loca
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go T

```

i-037c19b2f436f218e (theoryassign3ZainabQ)

PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

```

ubuntu@ip-172-31-17-82:~/webapp/app$ cd static
ubuntu@ip-172-31-17-82:~/webapp/app/static$ nano style.css
ubuntu@ip-172-31-17-82:~/webapp/app/static$ cd ..
ubuntu@ip-172-31-17-82:~/webapp/app$ cd templates
ubuntu@ip-172-31-17-82:~/webapp/app/templates$ nano index.html
ubuntu@ip-172-31-17-82:~/webapp/app/templates$ cd ..
ubuntu@ip-172-31-17-82:~/webapp/app$ ls
Dockerfile app.py requirements.txt static templates
ubuntu@ip-172-31-17-82:~/webapp/app$ 

```

i-037c19b2f436f218e (theoryassign3ZainabQ)

PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

4. Dockerization of the Application

4.1 Web Dockerfile

A Dockerfile was created under /app to containerize the Flask application. It installs all dependencies from requirements.txt and exposes port 5000.

4.2 MySQL Initialization Script

The database service uses init.sql to automatically create the tododb database and todos table when the container starts.

```
GNU nano 7.2 init.sql *
CREATE DATABASE IF NOT EXISTS tododb;

USE tododb;

CREATE TABLE IF NOT EXISTS todos (
  id INT AUTO INCREMENT PRIMARY KEY,
  task VARCHAR(255) NOT NULL
);

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Lc
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Gc
```

i-037c19b2f436f218e (theoryassign3ZainabQ)
PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

```
ubuntu@ip-172-31-17-82:~/webapp$ cd db
ubuntu@ip-172-31-17-82:~/webapp/db$ nano init.sql
ubuntu@ip-172-31-17-82:~/webapp/db$ ls
init.sql
ubuntu@ip-172-31-17-82:~/webapp/db$ cd ..
ubuntu@ip-172-31-17-82:~/webapp$
```

i-037c19b2f436f218e (theoryassign3ZainabQ)
PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82


```
GNU nano 7.2 test_unit.py *
import json
import pytest
from app import app

@pytest.fixture
def client():
    app.config["TESTING"] = True
    with app.test_client() as c:
        yield c

def test_add_task(client):
    r = Client.post("/api/todo", json={"task": "Test Task"})
    assert r.status_code == 201
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Loca
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To

i-037c19b2f436f218e (theoryassign3ZainabQ)
PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

```
GNU nano 7.2 test_selenium.py *
import time
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

def get_driver():
    opts = Options()
    opts.add_argument("--headless=new")
    opts.add_argument("--no-sandbox")
    opts.add_argument("--disable-dev-shm-usage")
    return webdriver.Chrome(options=opts)

def test_homepage_loads():
    d = get_driver()
    d.get("http://web:5000/")
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Locatio
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To

i-037c19b2f436f218e (theoryassign3ZainabQ)
PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

```
ubuntu@ip-172-31-17-82:~/webapp$ cd tests
ubuntu@ip-172-31-17-82:~/webapp/tests$ nano test_unit.py
ubuntu@ip-172-31-17-82:~/webapp/tests$ nano test_selenium.py
ubuntu@ip-172-31-17-82:~/webapp/tests$ ls
test_selenium.py  test_unit.py
ubuntu@ip-172-31-17-82:~/webapp/tests$ cd ..
ubuntu@ip-172-31-17-82:~/webapp$
```

i-037c19b2f436f218e (theoryassign3ZainabQ)
PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

```
GNU nano 7.2 Dockerfile *
FROM python:3.11-slim

RUN apt-get update && apt-get install -y \
    wget unzip gnupg2 curl \
    libnss3 libgconf-2-4 libxss1 \
    libatk-bridge2.0-0 libgtk-3-0 \
    --no-install-recommends

# Install Chrome
RUN wget -q -O - https://dl.google.com/linux/linux_signing_key.pub | apt-key add - && \
    echo "deb http://dl.google.com/linux/chrome/deb/ stable main" > /etc/apt/sources.list.d/google.list && \
    apt-get update && apt-get install -y google-chrome-stable

RUN pip install selenium chromedriver-binary==114.0.5735.90.0 pytest requests

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo      M-G

i-037c19b2f436f218e (theoryassign3ZainabQ)
PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82
```

```
GNU nano 7.2 run_tests.sh *
#!/bin/bash
echo "Sleeping until web container is up..."
sleep 10
pytest -q

```

```
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   ^/ Go To
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To
```

i-037c19b2f436f218e (theoryassign3ZainabQ)

PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

```
ubuntu@ip-172-31-17-82:~/webapp$ cd selenium-docker
ubuntu@ip-172-31-17-82:~/webapp/selenium-docker$ nano Dockerfile
ubuntu@ip-172-31-17-82:~/webapp/selenium-docker$ nano run_tests.sh
ubuntu@ip-172-31-17-82:~/webapp/selenium-docker$ ls
Dockerfile  run_tests.sh
ubuntu@ip-172-31-17-82:~/webapp/selenium-docker$ cd ,,
-bash: cd: ,,: No such file or directory
ubuntu@ip-172-31-17-82:~/webapp/selenium-docker$ cd ..
ubuntu@ip-172-31-17-82:~/webapp$
```

i-037c19b2f436f218e (theoryassign3ZainabQ)

PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

5. Docker Compose Setup

A docker-compose.yml file was created to orchestrate:

- **web** container (Flask app)
- **db** container (MySQL)
- **selenium_tests** container (for automated browser testing)

Docker Compose was used to build and start the application:

```
docker-compose up -d --build
```

The web app became accessible via:

<http://<EC2-Public-IP>:5000>

```
GNU nano 7.2                                docker-compose.yml *
```

```
version: "3.8"

services:

  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: tododb
    volumes:
      - mysql_data:/var/lib/mysql
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    ports:
      - "3306:3306"
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Lir

i-037c19b2f436f218e (theoryassign3ZainabQ)
PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

```
GNU nano 7.2                                Jenkinsfile *
```

```
pipeline {
  agent any

  environment {
    // change these if you want different image tags / registry
    WEB_IMAGE = "my-webapp_web:latest"
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }
  }
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Locati
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To

i-037c19b2f436f218e (theoryassign3ZainabQ)
PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

```

ubuntu@ip-172-31-17-82:~/webapp$ nano docker-compose.yml
ubuntu@ip-172-31-17-82:~/webapp$ nano Jenkinsfile
ubuntu@ip-172-31-17-82:~/webapp$ ls
Jenkinsfile  app  db  docker-compose.yml  selenium-docker  tests
ubuntu@ip-172-31-17-82:~/webapp$ █

```

i-037c19b2f436f218e (theoryassign3ZainabQ)

PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

```

ubuntu@ip-172-31-17-82:~/webapp$ docker-compose --version
docker-compose version 1.29.2, build unknown
ubuntu@ip-172-31-17-82:~/webapp$ docker-compose up -d --build
Creating network "webapp_default" with the default driver
Creating volume "webapp_mysql_data" with default driver
Pulling db (mysql:8.0)...
8.0: Pulling from library/mysql
023a182c62a0: Pull complete
4f78e34adfad: Pull complete
a2ed1082d9e2: Pull complete
c9ecfb07ed08: Pull complete
4f94eaa123bf: Pull complete
2a2d53254403: Pull complete
48ec49971d94: Pull complete
fdca9f583d44: Pull complete
abcf302dead6: Pull complete

```

i-037c19b2f436f218e (theoryassign3ZainabQ)

PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

```

Successfully built 8d86b81cb642
Successfully tagged webapp_web:latest
Creating webapp_db_1 ... done
Creating webapp_web_1 ... done
ubuntu@ip-172-31-17-82:~/webapp$ docker-compose ps

```

Name	Command	State	Ports
webapp_db_1	docker-entrypoint.sh mysqld	Up	0.0.0.0:3306->3306/tcp,:::3306->3306/t
webapp_web_1	python app.py	Up (health: starting)	0.0.0.0:5000->5000/tcp,:::5000->5000/t

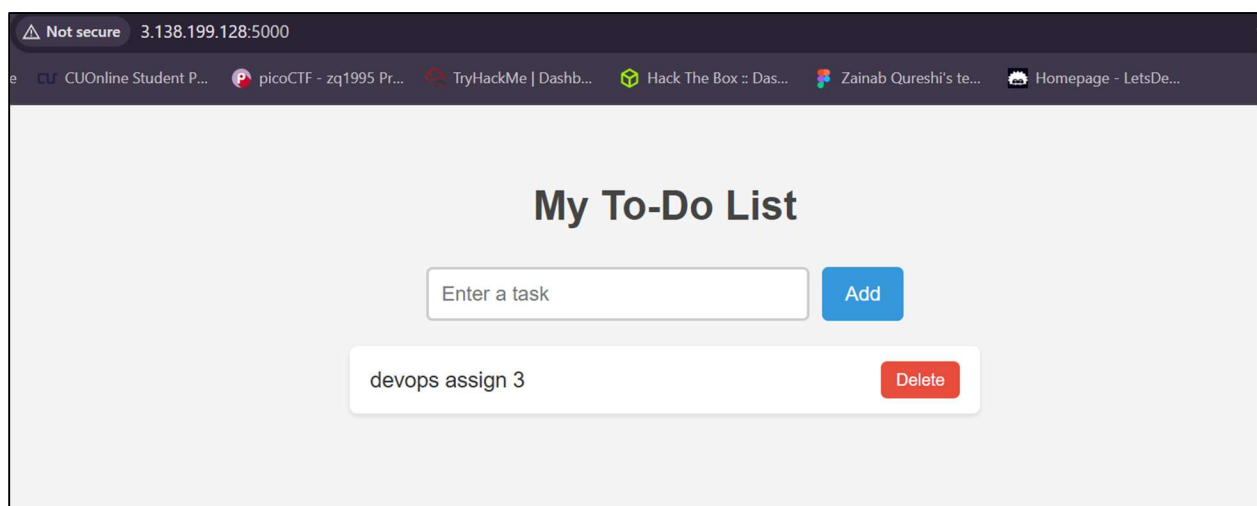
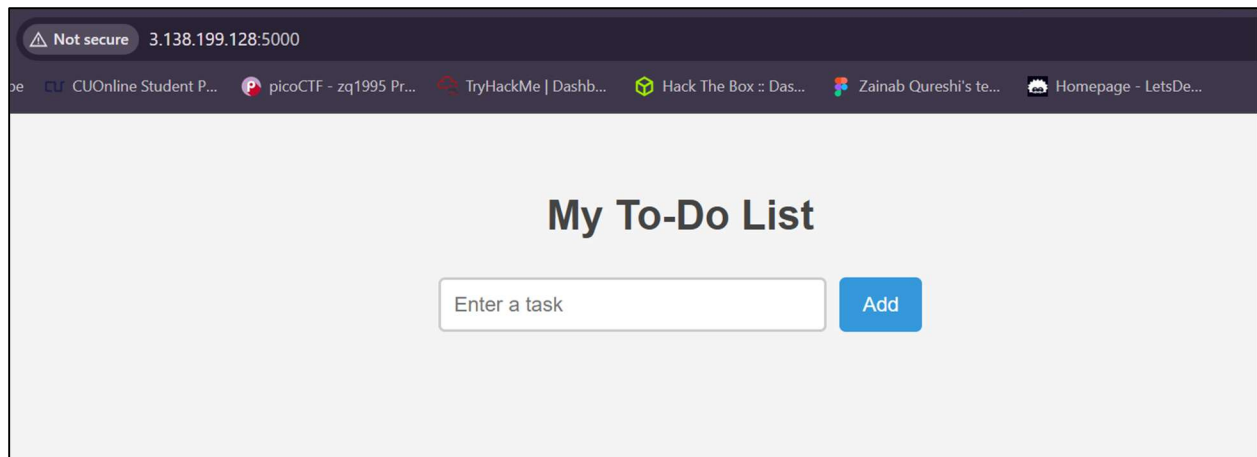
```

ubuntu@ip-172-31-17-82:~/webapp$ █

```

i-037c19b2f436f218e (theoryassign3ZainabQ)

PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82



6. GitHub Repository Setup

A new GitHub repository was created, and the entire project was pushed from the EC2 instance:

```
git init
```

```
git add .
```

```
git commit -m "Initial commit"
```

```
git remote add origin https://github.com/<username>/<repo>.git
```

```
git push -u origin master
```

A GitHub Personal Access Token (PAT) was used for authentication.

```
ubuntu@ip-172-31-17-82:~/webapp$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/ubuntu/webapp/.git/
ubuntu@ip-172-31-17-82:~/webapp$ git remote add origin https://github.com/zening/webapp.git
ubuntu@ip-172-31-17-82:~/webapp$ git add .
ubuntu@ip-172-31-17-82:~/webapp$ git commit -m "Initial commit - Flask, Docker, Selenium, Jenkins setup"
[master (root-commit) 18742eb] Initial commit - Flask, Docker, Selenium, Jenkins setup
Committer: Ubuntu <ubuntu@ip-172-31-17-82.us-east-2.compute.internal>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
```

i-037c19b2f436f218e (theoryassign3ZainabQ)

PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

```
ubuntu@ip-172-31-17-82:~/webapp$ git push https://zening:ghp_03a27f436f218e36f218e36f218e36f218e@github.com:zening/webapp.git HEAD:master
Enumerating objects: 20, done.
Counting objects: 100% (20/20), done.
Delta compression using up to 2 threads
Compressing objects: 100% (17/17), done.
Writing objects: 100% (20/20), 5.88 KiB | 5.88 MiB/s, done.
Total 20 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/zening/webapp.git
 * [new branch]      HEAD -> master
ubuntu@ip-172-31-17-82:~/webapp$
```

i-037c19b2f436f218e (theoryassign3ZainabQ)

PublicIPs: 3.138.199.128 PrivateIPs: 172.31.17.82

zening / webapp

Q Type to search

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

webapp

Public

Pin

Watch 0

master

1 Branch

0 Tags

Go to file

Add file

<> Code

Ubuntu

Initial commit - Flask, Docker, Selenium, Jenkins setup

18742eb · 3 minutes ago

1 Commit

app	Initial commit - Flask, Docker, Selenium, Jenkins setup	3 minutes ago
db	Initial commit - Flask, Docker, Selenium, Jenkins setup	3 minutes ago
selenium-docker	Initial commit - Flask, Docker, Selenium, Jenkins setup	3 minutes ago
tests	Initial commit - Flask, Docker, Selenium, Jenkins setup	3 minutes ago
Jenkinsfile	Initial commit - Flask, Docker, Selenium, Jenkins setup	3 minutes ago
docker-compose.yml	Initial commit - Flask, Docker, Selenium, Jenkins setup	3 minutes ago

7. Jenkins Installation & Configuration

A Jenkins server was installed and configured on the same EC2 instance.

Jenkins access was verified at:

`http://<EC2-Public-IP>:8080`

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

```
/var/lib/jenkins/secrets/initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

.....

Continue

8. GitHub Webhook Configuration

A webhook was added in the GitHub repository:

Repository → Settings → Webhooks

Payload URL: `http://<EC2-Public-IP>:8080/github-webhook/`

Content type: `application/json`

Trigger: Just the push event

This ensured Jenkins automatically triggered a pipeline on every commit.

Webhooks / Add webhook

We'll send a `POST` request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (`JSON`, `x-www-form-urlencoded`, *etc*). More information can be found in [our developer documentation](#).

Payload URL *

`http://3.138.199.128:8080/github-webhook/`

Content type *

`application/json`

9. Jenkins Pipeline (CI/CD) Implementation

A Jenkinsfile was written to automate the entire CI/CD workflow.

The pipeline performed the following stages:

9.1 Checkout Code

Automatically pulls the latest code from GitHub.

9.2 Linting

Runs Python linting inside a temporary Docker container using Flake8.

9.3 Build Docker Images

Builds the application and Selenium test images:

```
docker-compose build --no-cache
```

9.4 Unit Tests

Runs Python unit tests:

```
docker-compose run --rm web pytest -q
```

9.5 Integration + Selenium Tests

Starts the full application stack:

```
docker-compose up -d db web
```

Then executes Selenium browser tests from a dedicated test container.

9.6 Artifact Collection

Pipeline logs are saved into jenkins_artifacts/ for assignment submission.

9.7 Cleanup

All test containers, networks, and volumes are removed:

```
docker-compose down -v
```

New Item

Enter an item name

Select an item type



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

The Jenkins dashboard shows a build queue with no builds and a build executor status of 0/2. A table lists the build 'theoryassign3ZainabQ' with a status of 'Failed' (indicated by a sun icon) and 'Last Success' as 'N/A'.

S	W	Name	Last Success	Last Failure	Last Duration
...	☀	theoryassign3ZainabQ	N/A	N/A	N/A

The code editor shows the file 'trigger.txt' in the 'webapp' directory. The content of the file is:

```
1 this is the trigger for theory assign 3
```

The Jenkins dashboard shows the build 'theoryassign3ZainabQ' with a status of 'Success' (indicated by a checkmark icon) and 'Last Success' as '1 min 30 sec #5'.

S	W	Name	Last Success	Last Failure	Last Duration
✓	☁	theoryassign3ZainabQ	1 min 30 sec #5	3 min 39 sec #4	55 sec

10. Pipeline Execution and Success

Initially, the pipeline failed due to:

- Flake8 formatting errors
- Port conflicts with existing containers
- Missing cryptography dependency

- Docker command differences

These issues were resolved by:

- Fixing Python formatting
- Installing cryptography properly
- Updating the Jenkinsfile
- Ensuring no containers were already running during test stages

Finally, the pipeline executed **successfully**, completing all stages without errors.

Appendix:

Dockerfile for WebApp:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]
```

Db/init.sql:

```
CREATE DATABASE IF NOT EXISTS tododb;

USE tododb;

CREATE TABLE IF NOT EXISTS todos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    task VARCHAR(255) NOT NULL
);
```

Selenium Dockerfile:

```
FROM python:3.11-slim

# Install dependencies needed for Chrome + Selenium
RUN apt-get update && apt-get install -y \
    wget \
    unzip \
    curl \
    gnupg \
    libnss3 \
    libxss1 \
    libatk-bridge2.0-0 \
    libgtk-3-0 \
    libgbm1 \
    libasound2 \
    libu2f-udev \
    fonts-liberation \
    --no-install-recommends && \
```

```
rm -rf /var/lib/apt/lists/*

# Add Google Chrome signing key (modern method)
RUN mkdir -p /etc/apt/keyrings && \
    wget -qO- https://dl.google.com/linux/linux_signing_key.pub \
    | gpg --dearmor -o /etc/apt/keyrings/google-linux-signing-keyring.gpg

# Add Chrome repository
RUN echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/google-linux-signing-keyring.gpg] http://dl.google.com/linux/chrome/deb/ stable main" \
    > /etc/apt/sources.list.d/google-chrome.list

# Install Google Chrome
RUN apt-get update && apt-get install -y google-chrome-stable && \
    rm -rf /var/lib/apt/lists/*

# Install matching ChromeDriver
RUN CHROME_VERSION=$(google-chrome --version | awk '{print $3}') && \
    wget -O /tmp/chromedriver.zip https://storage.googleapis.com/chrome-for-testing-public/$CHROME_VERSION/linux64/chromedriver-linux64.zip && \
    unzip /tmp/chromedriver.zip -d /tmp/ && \
    mv /tmp/chromedriver-linux64/chromedriver /usr/bin/chromedriver && \
    chmod +x /usr/bin/chromedriver

# Install Selenium, pytest, requests
RUN pip install selenium pytest requests

# Add tests
WORKDIR /tests
COPY tests /tests
COPY selenium-docker/run_tests.sh /tests
RUN chmod +x /tests/run_tests.sh

CMD ["/tests/run_tests.sh"]
```

Selenium/run_tests.sh

```
#!/bin/bash
echo "Sleeping until web container is up..."
sleep 10
pytest -q
```

test_unit.py

```
import json
import pytest
from app import app

@pytest.fixture
def client():
    app.config["TESTING"] = True
    with app.test_client() as c:
        yield c

def test_add_task(client):
    r = client.post("/api/todo", json={"task": "Test Task"})
    assert r.status_code == 201

def test_get_tasks(client):
    r = client.get("/api/todo")
    assert r.status_code == 200
    assert isinstance(r.get_json(), list)
```

test_selenium.py

```
import time
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

def get_driver():
    opts = Options()
    opts.add_argument("--headless=new")
    opts.add_argument("--no-sandbox")
    opts.add_argument("--disable-dev-shm-usage")
    return webdriver.Chrome(options=opts)

def test_homepage_loads():
    d = get_driver()
    d.get("http://web:5000/")
    assert "To-Do" in d.page_source
    d.quit()

def test_add_task():
    d = get_driver()
    d.get("http://web:5000/")
```

```
input_box = d.find_element("id", "task-input")
input_box.send_keys("Selenium Task")

btn = d.find_element("css_selector", "button[type='submit']")
btn.click()

time.sleep(2)

assert "Selenium Task" in d.page_source
d.quit()
```

docker-compose.yml

```
version: "3.8"

services:

  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: tododb
    volumes:
      - mysql_data:/var/lib/mysql
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    ports:
      - "3306:3306"

  web:
    build: ./app
    depends_on:
      - db
    environment:
      DB_HOST: db
      DB_USER: root
      DB_PASS: password
      DB_NAME: tododb
    ports:
      - "5000:5000"
    # include a healthcheck so compose knows when web is ready
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request;
urllib.request.urlopen('http://localhost:5000')"]
      interval: 5s
```



```

    timeout: 3s
    retries: 12

selenium_tests:
  build:
    context: .
    dockerfile: ./selenium-docker/Dockerfile
  depends_on:
    web:
      condition: service_healthy
  profiles: ["test"]
  # When this service finishes, compose can exit (we use --exit-code-from)
  # The run script in selenium-docker/run_tests.sh should exit with pytest exit
code.
  environment:
    - PYTHONUNBUFFERED=1
  # don't restart - we want a single run that exits
  restart: "no"

volumes:
  mysql_data:

```

Jenkinsfile:

```

pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        git branch: 'master', url: 'https://github.com/zeninq/webapp.git'
      }
    }

    stage('Lint') {
      steps {
        echo "Skipping Python package installs – using Docker
environment"
        sh """
            docker run --rm \
            -v \$(pwd)/app:/app \
            python:3.11-slim \
            sh -c "pip install flake8 && flake8 /app"
            """
      }
    }
  }
}

```

```

    }
}

stage('Build images') {
    steps {
        sh "docker-compose build --no-cache"
    }
}

stage('Unit tests') {
    steps {
        sh """
            docker-compose run --rm web pytest -q || true
        """
    }
}

stage('Containerized integration + Selenium tests') {
    steps {
        sh """
            docker-compose up -d db web
            sleep 8
            docker-compose run --rm selenium_tests || true
        """
    }
}

}

post {
    always {
        echo "---- COLLECTING LOGS ----"
        sh """
            mkdir -p jenkins_artifacts/logs
            docker-compose logs > jenkins_artifacts/logs/compose_logs.txt ||
true
        """
        archiveArtifacts artifacts: 'jenkins_artifacts/logs/**/*',
allowEmptyArchive: true

        echo "---- CLEANUP ----"
        sh "docker-compose down -v || true"
    }
    failure {
        echo "Pipeline failed."
    }
}

```

```
}  
}
```